

Deliverable: Continuous Integration

Cohort 1, Team 8

Team Members:

Michael Coleman

Finn Done

Charlie Webster

Philip McGowan

Hammad Sami

Tolly Shearman

Eric Vokes

Summary Of Continuous Integration Method(s) & Approach(es)

Continuous integration (CI) is a software development technique in which team members integrate their work on a regular basis. Typically, each team member integrates at least once every day, resulting in several integrations every day. This was crucial in our project as if we integrated everything last minute it could've resulted in crashes, a broken demo and stress that should never have been there.

Our team adopted a continuous integration approach aligned with our Agile, iterative and incremental development lifecycle. Continuous integration in our project meant that code was merged into a shared main repository frequently rather than at the end of development. This reduced the risk of large, late integration failures and helped us maintain a working version of the system throughout development. Having a shared repository meant there was a lot of transparency and everyone could monitor what was happening through the repository action tab which was important in the process because we would be immediately alerted when a new code commit breaks the build or fails a test (Rapid Issue Resolution).

We followed key CI practices from the course material, including maintaining a single shared source repository, committing small increments, and ensuring that the main branch always built successfully. Our team worked on short-lived feature branches that were merged through peer-reviewed pull requests. Each merge required the project to compile successfully, acting as a build gate to prevent broken code entering the mainline. This created fast feedback as integration problems were detected soon after they were introduced.

The CI process was tightly connected to our Scrum workflow. At the end of each sprint we aimed to have an integrated version of the game that could run and be demonstrated. Integration was not left to the end to do and it occurred continuously throughout iterations. This supported incremental delivery of features such as new game mechanics, assets, and UI changes.

A task wasn't considered finished just because the code was written, it was only done once it had been merged into the main branch and the project built successfully. We also tried to make builds reproducible by agreeing on tool versions. To keep integration simple, we kept branches short-lived and changes small and we tagged stable sprint releases so there was always a version we could go back to. After merges, we ran quick checks to make sure the game actually launched.

This CI approach was particularly appropriate for a small-scale project. It provided the advantages of early defect detection, shared visibility of progress and reduced merge conflicts whilst also avoiding the overhead of industrial-scale pipelines. It also matched the skill level of the team and the time constraints of the module, offering a realistic but achievable application of professional CI practice.

Brief Report On The Continuous Integration Infrastructure

Our continuous integration infrastructure was built around a single shared GitHub repository, which acted as the project's mainline. All team members committed to this central main branch or to short-lived branches that were merged back frequently. The mainline was treated as the authoritative, always-buildable version of the project.

We used Git and GitHub to support mainline development with frequent integration rather than infrequent merging. Changes were committed in small increments and integration happened early and often so that conflicts were found while they were still easy to resolve. At the end of each Scrum sprint, we ensured that the mainline could be built and run, corresponding to the CI principle that the mainline should always be in a working state.

Automation was supported using GitHub Actions which provided an automated build workflow. On each push or pull request, the project was automatically checked out and built. This gave the team rapid feedback when integrations failed and reduced the likelihood of broken code remaining in the main branch. Where appropriate, smoke tests were used to verify that the game launched successfully, not only that it compiled.

When a build failed, we followed the CI rule of fixing broken mainlines immediately. Further work paused until the error was corrected, preventing additional changes from being layered onto a non-working build. Communication about failures and fixes took place through Discord to ensure fast coordination between team members.

Additionally, the project was simpler to replicate and comprehend thanks to our structure. In order to prevent the typical issues, we decided that all team members would use the same tool versions and JDK. Everyone could see whether the main branch was healthy thanks to GitHub's build status and tagging sprint releases provided us with dependable versions to revert to in the event of a serious issue. Together these decisions improved our CI infrastructure beyond simple version control and enabled us to maintain consistent operation of the main branch.

Overall, the infrastructure put the core CI principles such as a single shared mainline, short-lived branches, automated builds, frequent integration, rapid feedback, tagged stable versions and an always-buildable main branch into practice. It delivered an appropriate level of automation and process control for a small project, while still closely reflecting professional CI practice.